CSC 255

Unit 4 Programming Assignment 2 [60 points]

Extendible Hashing

Read about extendible hashing from the links and documents provided in this Unit.

Write a program to implement extendible hashing using a dynamic array data structure such that each index (i.e., n-bit directory where $D = n$) maintains a link to a fixed size array (i.e., leaf of size M) which stores its associated items.

We will call this data structure an `ExtendibleHashTable`.

Include functionality for displaying the table, and inserting, finding, and removing items. **[20 points]**

Show the result of inserting the items: 10111101, 00000010, 10011011, 10111110, 01111111, 01010001, 10010110, 00001011, 11001111, 10011110, 11011011, 00101011, 01100001, 11110000, 01101111, 00000101, 01000101, 01000000 into an initially empty `ExtendibleHashTable` with $M = 4$. **[10 points]**

Also, show the result of inserting the items 16, 4, 6, 22, 24, 10, 31, 7, 9, 20, 26, 3, 1 into an initially empty `ExtendibleHashTable` with $M = 3$. **[10 points]**

Here is a representation of the resulting table for each of these sets of items (**Figure 1** and **Figure 2**):

Directory Index	Depth	Reference Dirs	Alias Of	Items (Binary → Decimal)
000	2	{0, 4}	—	11110000 (240), 01000000 (64)
001	3	{1}	—	01010001 (81), 01100001 (97)
010	2	{6}	**110**	—
011	3	{3}	—	10011011 (155), 00001011 (11), 11011011 (219), 00101011 (43)
100	2	{0}	**000**	—
101	3	{5}	—	10111101 (189), 00000101 (5), 01000101 (69)
110	2	{2, 6}	—	00000010 (2), 10111110 (190), 10010110 (150), 10011110 (158)
111	3	{7}	—	01111111 (127), 11001111 (207), 01101111 (111)

Figure 1

Directory Index	Depth	Reference Dirs	Alias Of	Items (Binary → Decimal)
000	3	{0}	—	00010000 (16), 00011000 (24)
001	2	{1, 5}	—	00001001 (9), 00000001 (1)
010	3	{2}	—	00001010 (10), 00011010 (26)
011	2	{7}	**111**	—
100	3	{4}	—	00000100 (4), 00010100 (20)
101	2	{1}	**001**	—
110	3	{6}	—	00000110 (6), 00010110 (22)
111	2	{3, 7}	—	00011111 (31), 00000111 (7), 00000011 (3)

Figure 2

The initial size of the `ExtendibleHashTable` is two (2) so that it can store an item (i.e., value) that hashes to a binary representation (i.e., key) ending in a 0 or a 1 (here, $D = 1$). The table will dynamically grow (i.e., extend) as the items are inserted and the dynamic nature of the splitting of leaves should also occur as necessary.

Insert 500 uniformly distributed, random, unsigned 64-bit integers (i.e., the keys) into the extendible hash table. In this case $N = 500$ and each integer would be converted to its binary form. Build the extendible hash table using $M = 4$ and $D = 6$; and, using $M = 8$ and $D = 7$. Compare the size of each directory and write a 100 to 200-word summary of your findings. Your summary should consider the actual number of leaves, the expected number of leaves which is $(N/M) \cdot log_2 e$, the number of times the keys are rehashed into new directories, and the number of duplicate keys, if any. **[20 points]**

An extendible hash table could be used to organize items sold at a store where each item has an associated item number. Each item number will be hashed to its respective leaf where the details of the item will be stored.

For example, item number: 568-39-1453 (i.e., 568391453) gets hashed to directory 53 where the details of the item are then stored (**Figure 3**).

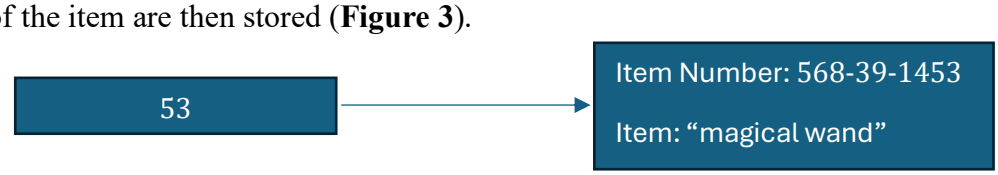

Figure 3

Use the following pseudocode to build an extendible hash table data structure:

```
CLASS Leaf:
    METHOD Initialize(index, depth):
        --Initialize the leaf node with the given index and depth
        self.index = index
        self.depth = depth
        self.directories = empty set    --Tracks directories referencing this leaf
        self.items = empty list    --Stores items in the leaf

    METHOD ToString():
        --Return a string representation of the leaf
        str_representation = "depth: " + ConvertToString(self.depth) + ", "
        str_representation += "reference dirs: "
        IF Size(self.directories) == 0 THEN
            str_representation += "None"
        ELSE
            str_representation += ConvertToString(self.directories)
        END IF
        str_representation += ", items: "
        FOR item IN self.items DO
            str_representation += "(" + item + ", "
                                + ConvertToString(BinaryToDecimal(item)) + "), "
        END FOR
        RETURN str_representation
END CLASS
CLASS ExtendibleHashTable:
    METHOD Initialize(globalDepth = 1, maxBucketSize = 4):
        --Initialize the hash table with default depth and bucket size
        self.globalDepth = globalDepth
        self.maxBucketSize = maxBucketSize
        self.directory = Dictionary()    --Map indices to Leaf objects
```

```
        --Initialize 2^globalDepth entries in the directory
        FOR i FROM 0 TO (2^globalDepth) - 1 DO
            self.directory[i] = New Leaf(i, globalDepth)
        END FOR
        --Initialize counters
        self.rehashCount = 0
        self.duplicateCount = 0
    METHOD Insert(key):
        --Insert a key-value pair into the hash table.
        --key: bit string representation
    METHOD Split(leaf):
        --Split the bucket when it exceeds maxBucketSize
    METHOD DoubleDirectorySize():
        --Double the directory size by duplicating entries
    METHOD UpdateDirectory(leaf, newLeaf):
        --Update the directory to point to the new leaf
    METHOD Search(key):
        --Search for a key in the hash table
END CLASS
```

Let's observe the output of inserting the following items (from above) into an extendible hash table:

{"10111101", "00000010", "10011011", "10111110", "01111111", "01010001", "10010110", "00001011", "11001111", "10011110", "11011011", "00101011", "01100001", "11110000", "01101111", "00000101", "01000101", "01000000"}

Here is a sample run showing the extendible table after inserting the items, results of finding and removing items; and, the extendible table after removing some of the items:

```
Directory index: 000, items: depth: 2, reference dirs: {0, 4}, items: 11110000 (240), 01000000 (64)
Directory index: 001, items: depth: 3, reference dirs: {1}, items: 01010001 (81), 01100001 (97)
Directory index: 010, alias of 110, depth: 2, reference dir: {6}
Directory index: 011, items: depth: 3, reference dirs: {3}, items: 10011011 (155), 00001011 (11), 11011011 (219), 00101011 (43)
Directory index: 100, alias of 000, depth: 2, reference dir: {0}
Directory index: 101, items: depth: 3, reference dirs: {5}, items: 10111101 (189), 00000101 (5), 01000101 (69)
Directory index: 110, items: depth: 2, reference dirs: {2, 6}, items: 00000010 (2), 10111110 (190), 10010110 (150), 10011110 (158)
Directory index: 111, items: depth: 3, reference dirs: {7}, items: 01111111 (127), 11001111 (207), 01101111 (111)
Used indices (unique leaves): 6
D: 3
# of expansions: 2
# of duplicates: 0

Found Item: 00000101 (5), at leaf depth: 3, is found at directory: 101
Found Item: 10111110 (190), at leaf depth: 2, is found at directory: 110
Found Item: 10111101 (189), at leaf depth: 3, is found at directory: 101
Remove Item: 10111101 (189)
Item: 10111101 (189): Key not found.
Found Item: 00001011 (11), at leaf depth: 3, is found at directory: 011
Found Item: 00000010 (2), at leaf depth: 2, is found at directory: 010
Remove Item: 00000101 (5)
Remove Item: 01000101 (69): leaf emptied; cleared pointers [5]
Remove Item: 00000000 (0): Key not found.
Remove Item: 11110000 (240)
Remove Item: 01000000 (64): leaf emptied; cleared pointers [0, 4]

Directory index: 000, items: None
Directory index: 001, items: depth: 3, reference dirs: {1}, items: 01010001 (81), 01100001 (97)
Directory index: 010, alias of 110, depth: 2, reference dir: {6}
Directory index: 011, items: depth: 3, reference dirs: {3}, items: 10011011 (155), 00001011 (11), 11011011 (219), 00101011 (43)
Directory index: 100, items: None
Directory index: 101, items: None
Directory index: 110, items: depth: 2, reference dirs: {2, 6}, items: 00000010 (2), 10111110 (190), 10010110 (150), 10011110 (158)
Directory index: 111, items: depth: 3, reference dirs: {7}, items: 01111111 (127), 11001111 (207), 01101111 (111)
Used indices (unique leaves): 4
D: 3
# of expansions: 2
# of duplicates: 0
```

Notice each directory identifies the other directories that reference them.